

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
TRUONG DAI HOC KHOA HOC VA CONG NGHE HA NOI

Bachelor Internship Report

Design and Evaluation of an Observability System
for AI-Assisted PCB Inspection
Using Grafana Loki

Student: Do Hung Anh
Student ID: 23BI14015
Program: Information and Communication Technology Program
External Supervisor: Eng. Pham Xuan Long
Internal Supervisor: Dr. Kieu Quoc Viet
Host Organization: FUYU Precision Component VN (Foxconn Industrial Internet)

Hanoi, 2026

Contents

1	Introduction	1
1.1	Problem Context: What AOI Monitoring Must Support	2
1.2	Research Framing: What, Why, How, and Results	2
1.3	Objectives	3
1.4	Why This Work Is a Suitable Bachelor Internship Topic	3
2	Related Work	3
2.1	Observability in Software Systems	3
2.2	Monitoring Stacks Based on Logs	4
2.3	ML and MLOps Monitoring	4
2.4	Monitoring in Industrial Inspection	4
2.5	Comparison With Large-Scale Monitoring Platforms	4
3	System Design	5
3.1	Design Principles	5
3.2	Architecture Overview	6
3.3	Event Model	6
3.4	Dashboard Logic	7
3.5	Anomaly Types	7
3.6	Why This Design Pattern Was Chosen	7
3.7	MLOps and Observability-as-Code Patterns	8
4	Case Study Implementation	8
4.1	Repository-Aligned AOI Platform	8
4.2	Implemented Monitoring Components	9
4.3	Event Flow in the AOI Service	9
4.4	Why the AOI Project is a Good Monitoring Case	9
4.5	Code Pattern: Event Construction and Safe Emission	9
5	Experiments	10
5.1	Experimental Goal	10
5.2	Experiment Design	10
5.3	Experiment Harness: Build, Run, and Report	11
5.4	Scenario Set	12
5.5	Representative Scenario Evidence	12
5.6	Additional Scenario Coverage	13
5.7	Measurements	14
5.8	Representative LogQL Queries	14
5.9	Measured Results Summary	16
5.10	Observed Outcome	16
6	Evaluation and Discussion	20
6.1	Evaluation Criteria	20
6.2	Evaluation Logic	20
6.3	Strengths of the Proposed Approach	20
6.4	Findings From the Executed Scenario Suite	20
6.5	Detection Latency and Alert Behavior	21

6.6	Limitations	21
6.7	Future Work	22
6.8	Chapter Conclusion	23

Design and Evaluation of an Observability System for AI-Assisted PCB Inspection Using Grafana Loki

Bachelor Internship Report

Abstract

This report presents the design and evaluation of an observability system for an AI-assisted printed circuit board (PCB) inspection service using Grafana Loki. The work is motivated by an industrial Automatic Optical Inspection (AOI) workflow, where understanding how an AI inference service behaves during operation is as important as the prediction results themselves.

To investigate this problem, a case study based on a PCB AOI platform was developed. The system consists of a FastAPI backend responsible for generating and managing structured inference events, a React-based workstation for operator review, and a Loki–Promtail–Grafana stack used to collect, store, query, and visualize operational data. Through this architecture, inference events can be traced, analyzed, and monitored in near real time.

The monitoring approach focuses on structured JSON logging, defect-level traceability, latency tracking, historical log analysis, and anomaly detection under different operating conditions. A series of controlled experiments was conducted to inject known anomaly patterns representing normal operation, failure scenarios, and degraded inference performance. Because the inference events are generated synthetically, the evaluation validates the observability pipeline rather than the predictive accuracy of the underlying detector: the results show that a lightweight stack can reliably surface the injected conditions—fail-rate spikes, latency degradation, low-confidence inputs, and gradual drift—and preserve queryable evidence to support debugging, evaluation, and future system improvements.

Keywords: AI monitoring, AOI, PCB inspection, inference logging, Loki, Grafana, Promtail, FastAPI, observability, industrial AI

1 Introduction

Automatic Optical Inspection (AOI) systems are widely used in electronics manufacturing to inspect Printed Circuit Boards (PCBs) for defects such as missing components, misalignment, solder bridges, and insufficient solder. Modern AOI workflows increasingly incorporate AI-based models to classify or localize potential defects automatically, helping operators process inspection results more efficiently.

While model accuracy remains important, deploying AI in a production workflow introduces additional operational challenges. Engineers need to understand not only what the model predicts, but also how the inference service behaves during runtime. High latency, unstable responses, missing records, or abnormal prediction patterns can reduce the reliability of an inspection system even when the model itself performs well.

During an AOI workflow, inference results are connected to multiple stages including image acquisition, inspection management, defect review, and operator decisions. For

this reason, prediction outputs alone are often insufficient. Additional information such as timestamps, board identifiers, confidence scores, inference latency, and review outcomes is required to support troubleshooting, auditing, and performance analysis.

This report investigates how observability techniques can be applied to an AI-assisted AOI system. The proposed approach treats inference events as operational data and records them through structured logging. Instead of using logs only for debugging, the system generates machine-readable events that describe what happened, when it happened, and how the inference service responded.

To evaluate this approach, a repository-aligned AOI platform was developed using FastAPI, React, JSONL event logging, and a Loki–Promtail–Grafana observability stack. The monitoring system is designed to support real-time observation, historical log analysis, latency tracking, and anomaly detection under different operating conditions. The goal is to demonstrate that AI inference services can be monitored and evaluated as operational systems rather than treated solely as prediction engines.

1.1 Problem Context: What AOI Monitoring Must Support

For this report, the practical object under study is not only the PCB detector itself, but the operational layer around it. In an AOI environment, a useful monitoring system should support at least four needs:

- visibility into how many inference events are being produced;
- traceability of which PCB or component triggered a failure;
- observability of latency, confidence, and abnormal prediction trends; and
- support for operator review and post-incident investigation.

Without these needs, an AOI system may still generate predictions, but it remains difficult to trust, debug, or operate reliably in an industrial environment.

1.2 Research Framing: What, Why, How, and Results

This report is structured around four questions.

What is being built? An AI inference logging and monitoring system for a simulated PCB AOI workflow, using structured event logging and the Loki–Promtail–Grafana stack.

Why is it needed? Because AI-assisted inspection systems require operational visibility in addition to model accuracy. Engineers and operators need real-time monitoring, historical log search, and anomaly detection to maintain reliability.

How is it implemented? Through a case study that combines a FastAPI backend, a React review workstation, JSONL event logs, Promtail for log shipping, Loki for aggregation and querying, and Grafana for dashboards.

What results are expected? The system makes it possible to distinguish healthy behavior from failure spikes and latency degradation, while also preserving traceable evidence for evaluation and debugging.

1.3 Objectives

The objectives of this report are as follows:

- Explain why inference monitoring matters in operational AI systems;
- Review observability practices relevant to ML services;
- Design a monitoring architecture suitable for a local AOI platform;
- Describe a concrete implementation grounded in the PCB AOI project;
- Propose experiments that simulate multiple runtime conditions; and
- Evaluate whether the system supports practical anomaly detection.

1.4 Why This Work Is a Suitable Bachelor Internship Topic

This topic is appropriate for a bachelor internship because it combines several important skills in one applied engineering problem: backend integration, structured data design, industrial software workflow understanding, dashboard construction, and system evaluation. The work is also concrete and contained. It does not attempt to solve all of AOI or all of machine learning. Instead, it focuses on a realistic and defensible contribution: making an AI-based AOI service observable and evaluable in practice.

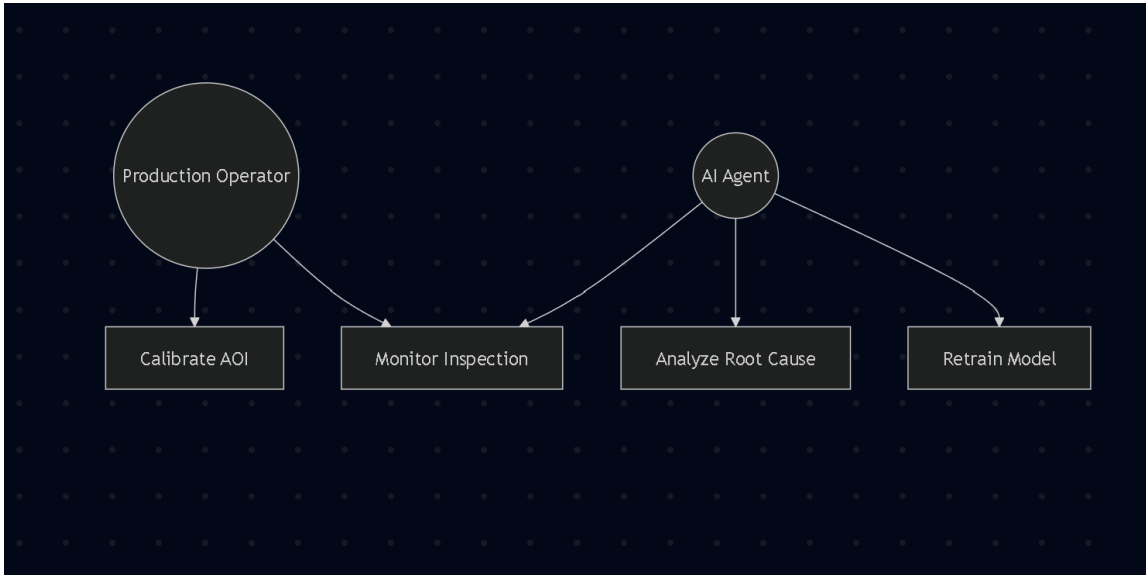


Figure 1: Conceptual AOI monitoring loop linking operator decisions, inspection monitoring, root-cause analysis, and model improvement.

2 Related Work

2.1 Observability in Software Systems

Observability is a core idea in distributed systems and production software [1]. Traditional monitoring emphasizes aggregate counters and infrastructure health, while observability

aims to make internal system behavior understandable through logs, metrics, and traces. In AI systems, this distinction is important because model-serving failures may appear even when CPU, memory, or network indicators seem normal.

2.2 Monitoring Stacks Based on Logs

The Loki stack is a pragmatic logging solution for lightweight deployments [8]. Loki aggregates logs, Promtail ships them from services, and Grafana provides queries, dashboards, and alert-friendly visualization [2]. Compared with heavyweight full observability platforms, this stack is attractive for report-scale or prototype environments because it is easy to deploy locally and integrates well with structured JSON logs.

2.3 ML and MLOps Monitoring

Monitoring in machine learning systems usually covers several categories: system health, inference latency, prediction distribution, data drift, model versioning, and human feedback loops. Large MLOps platforms often include these features [10], but smaller projects frequently need a narrower and more direct approach. For a local AOI workstation, the most relevant signals are event volume, board throughput, failure frequency, confidence patterns, and per-event latency.

This report is also informed by a broader software-engineering literature on ML systems. Sculley et al. [3] argue that production ML systems accumulate system-level technical debt through entanglement, hidden feedback loops, undeclared consumers, and brittle interfaces rather than only through poor model performance. Breck et al. [4] propose the ML Test Score as a production-readiness rubric, emphasizing data validation, infrastructure testing, and monitoring discipline around deployed models. Amershi et al. [5] show, through an industrial case study at Microsoft, that developing ML-enabled products requires a wider process change than conventional software engineering, especially around data, versioning, and integration. Shankar et al. [6, 7] extend this discussion by arguing that production ML pipelines require end-to-end observability because silent failures often occur outside the model itself. These works strengthen the motivation for a thesis that treats AOI inference as an operational system to be monitored, not only a predictor to be benchmarked.

2.4 Monitoring in Industrial Inspection

Industrial inspection systems differ from generic web services because they are tightly connected to physical workflows. Missing logs can make defect review non-reproducible, while unstable latency can slow operator decisions. This means monitoring must support both engineering needs and operational review. This requirement mirrors the observability and MLOps principles discussed above [1, 6]: trustworthy inspection requires end-to-end traceability of each inference event.

2.5 Comparison With Large-Scale Monitoring Platforms

Large industrial AI systems often operate at a scale far beyond a bachelor internship project, and it is useful to state this difference clearly. Netflix reports that its Mantis platform has been in production since 2014 and processes trillions of events and peta-bytes

of data every day [11]. The same public documentation describes use cases that analyze data from thousands of servers, millions of interactions across dozens of microservices, and hundreds of Cassandra and Elasticsearch clusters [12]. In separate Netflix platform material, the company also reports tens of thousands of running instances across its deployment environment [13].

This report does not attempt to replicate that scale. Instead, it adopts a similar engineering principle at a much smaller scope: preserve structured raw events, keep the monitoring pipeline queryable in near real time, and make it possible to ask new operational questions without changing the application each time a new anomaly is suspected.

It is also useful to compare the present design with specialized inference servers such as NVIDIA Triton. Triton exposes a rich metrics surface including request duration, queue duration, inference count, and execution count [14]. Those capabilities are important in high-throughput GPU-serving environments. By contrast, the AOI workstation in this report is optimized for local deployability, defect traceability, and end-to-end observability of an AOI review workflow rather than raw serving throughput. The tradeoff is deliberate: FastAPI plus structured JSONL logging [9] is easier to audit, easier to modify, and better matched to a repository-scale case study than a full production-grade inference platform.

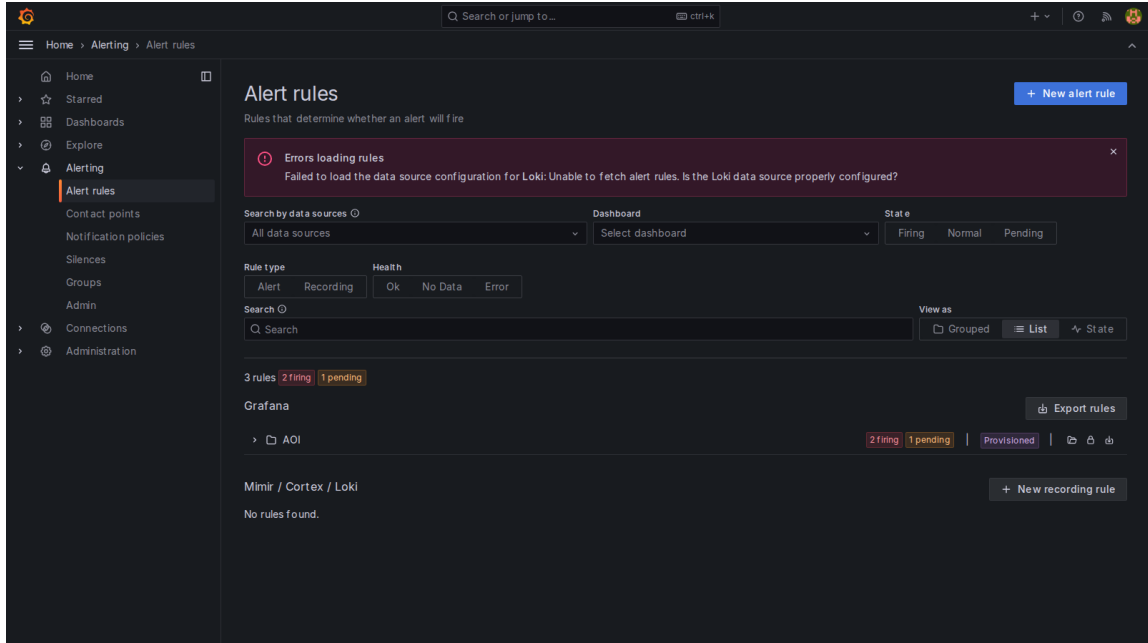


Figure 2: Provisioned alert rules demonstrate the transition from passive observability to active monitoring.

3 System Design

3.1 Design Principles

The proposed monitoring architecture follows five principles:

1. Structured logs instead of free-form strings;
2. Event schemas aligned with AOI domain entities;

3. Separation between inference, persistence, and visualization;
4. Local deployability for development and report demonstration; and
5. Support for anomaly detection through simple, inspectable queries.

3.2 Architecture Overview

The architecture begins with the AOI application, which emits inference events as structured JSON entries. Each event may include a PCB identifier, component identifier, inspection result, defect type, confidence score, latency, and normalized overlay geometry. These logs are written to a file that Promtail scrapes and forwards to Loki. Grafana then queries Loki to present operational dashboards such as total events, recent failures, boards processed, and latency trends.

In repository terms, the path is concrete rather than conceptual. A FastAPI API accepts `POST /events` requests, writes event rows to JSONL, and stores run metadata in SQLite. Promtail scrapes `/var/log/aoi/*.jsonl`, extracts selected fields from each JSON object, and promotes a subset of those fields to labels. Loki stores the resulting log streams, while Grafana provides the dashboard and alerting surface. This design keeps the event source, log transport, query model, and visual layer clearly separated. End to end, an event becomes visible on the dashboards within a few seconds: Promtail tails the JSONL files continuously and forwards them to Loki within roughly a second, and the Grafana panels refresh on a 5-second interval. This 5-second refresh is the practical bound on the “near real time” freshness referred to throughout this report.

3.3 Event Model

The event schema is the core monitoring contract. An effective schema should capture both model output and operational metadata. In the AOI case, the most important fields are:

- `timestamp` for temporal ordering;
- `pcb_id` for board-level grouping;
- `component_id` for defect localization;
- `inspection_result` for pass/fail state;
- `defect_type` for defect categorization;
- `confidence_score` for prediction strength;
- `inference_latency_ms` for performance monitoring; and
- overlay coordinates for frontend review traceability.

This field selection is intentionally narrower than a full cloud MLOps schema. The goal is to preserve the signals most relevant to AOI operation while keeping queries simple. In the Promtail pipeline, `pcb_id`, `component_id`, `inspection_result`, and `defect_type` are promoted to labels because they are frequently used for filtering and grouping. By contrast, `confidence_score` and `inference_latency_ms` remain JSON values that are parsed on demand through `LogQL | json | unwrap ...` expressions. This reduces label cardinality while preserving numeric analysis.

3.4 Dashboard Logic

The dashboard is designed around operational questions:

- How many events arrived in the last five minutes?
- How many recent detections were failures?
- How many boards were processed in a recent time window?
- Is inference latency stable or increasing?

These questions map directly to Loki queries and make the dashboard useful for both live monitoring and retrospective investigation.

3.5 Anomaly Types

The design groups anomalies by the operational signal that changes: traffic volume, prediction outcomes, latency, confidence, or domain identifiers such as `pcb_id` and `defect_type`. This compact taxonomy determines which fields must be logged and which dashboard queries must be supported; the specific test scenarios are defined once in Chapter 5.

3.6 Why This Design Pattern Was Chosen

The monitoring architecture follows a log-centric MLOps pattern for four practical reasons.

1. **Low integration cost.** A FastAPI service can emit structured JSON without introducing a specialized model server or a metric exporter at the first stage of the project.
2. **High diagnostic value.** A single event record can carry both operational fields such as latency and AOI-specific fields such as `pcb_id` and `defect_type`. This is harder to capture in aggregate metrics alone.
3. **Observability as code.** Promtail parsing rules, Grafana dashboards, and alert thresholds are provisioned from version-controlled files, which makes the monitoring behavior reproducible.
4. **Fit to scope.** The thesis question is whether anomalies in an AOI inference service become visible and actionable. It is not whether the repository can outperform industrial serving platforms on throughput.

The main alternative would be a production inference stack centered on systems such as Triton, KServe, or a larger stream-processing platform. Those architectures are stronger when dynamic batching, GPU multiplexing, autoscaling, or multi-tenant serving are the central problem. In this project, however, the critical problem is explainable monitoring around a bounded AOI workflow. The chosen pattern therefore prioritizes interpretability over maximum serving complexity.

3.7 MLOps and Observability-as-Code Patterns

Although the implementation is lightweight, it still follows several recognizably MLOps-oriented patterns:

- **Schema as contract:** the event structure standardizes what an inference result must contain before it enters the monitoring stack.
- **Logs as canonical evidence:** every experiment ultimately becomes a sequence of JSONL records that can be replayed, queried, and audited.
- **Infrastructure as code:** alert rules and scrape pipelines are defined in YAML rather than configured manually in a UI.
- **Experiment harness separation:** scenario generation, sequential execution, and report extraction are implemented as separate modules rather than one ad hoc script.

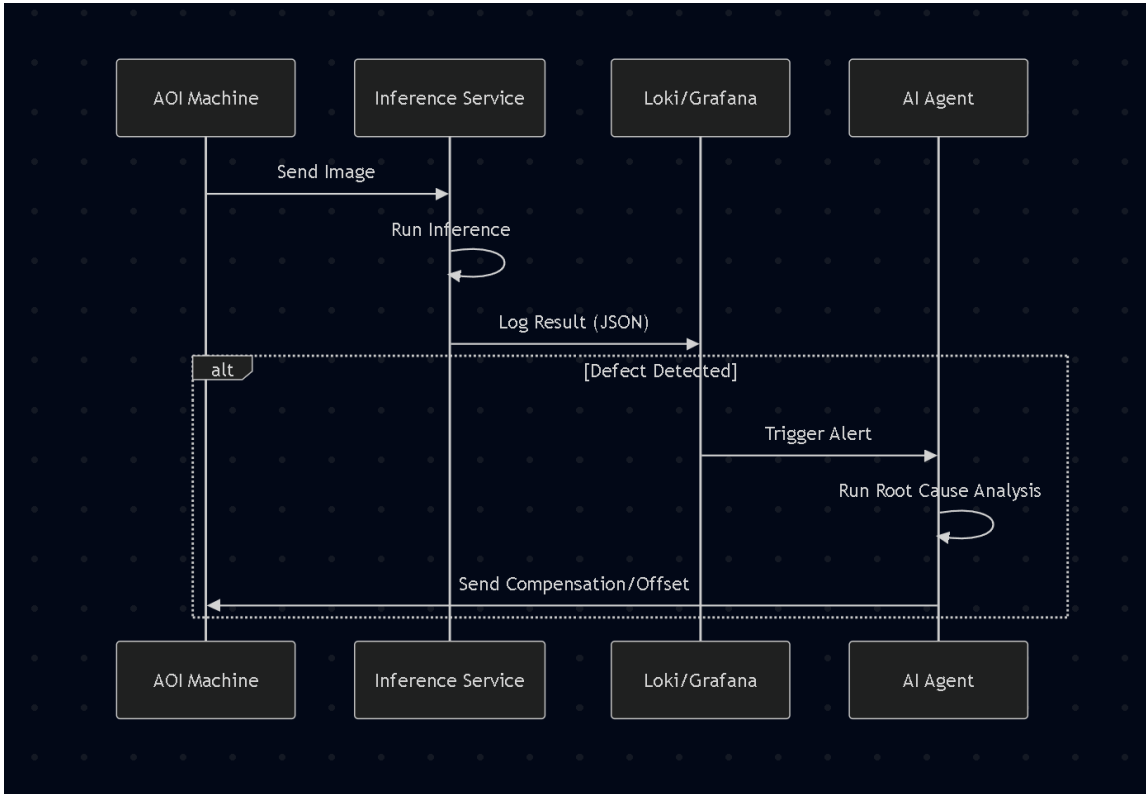


Figure 3: Repository-aligned architecture for structured event logging and anomaly monitoring.

4 Case Study Implementation

4.1 Repository-Aligned AOI Platform

The case study implementation is based on the current AOI repository. The backend is implemented in Python with FastAPI and exposes routes for health, runs, and event

ingestion. The frontend is a React workstation for PCB review, including run navigation, image viewing, overlay display, and operator review controls. SQLite is used for local persistence, and JSONL is used for event log output.

4.2 Implemented Monitoring Components

The implementation contains the following concrete monitoring-relevant modules:

- a Schema layer defining `InferenceEvent` and inspection states;
- a Log manager that appends JSON records to persistent log files;
- a Mock inference generator for controlled event simulation;
- a Docker Compose stack containing Loki, Promtail, and Grafana; and
- a Provisioned Grafana dashboard for AOI event inspection.

Two implementation details are especially important. First, the Promtail configuration parses JSON and promotes domain identifiers to labels. This is what makes queries such as `{job="aoi-inference", pcb_id= "PCB-DEAD-.*"}` possible without full-text search. Second, the alert rules are provisioned as code in Grafana, with three explicit threshold policies: fail rate > 0.60 , average latency > 500 ms, and average confidence < 0.60 . Each rule follows the same pattern: a Loki range query, a reduce stage, and a threshold stage.

4.3 Event Flow in the AOI Service

The service receives or generates inference-like outputs and converts them into structured events. A typical event includes board identity, component identity, inspection result, defect type, confidence score, latency, and optional defect overlay coordinates. The backend can then persist the event and expose related data to the review workstation. This creates a direct path from AI output to human inspection and to observability tooling.

4.4 Why the AOI Project is a Good Monitoring Case

The AOI project is appropriate for monitoring research because it combines image processing, event logging, persistence, operator review, and dashboarding within a single system. It also includes synthetic event generation, which is useful for testing failure scenarios without requiring a fully deployed production model. This makes the platform suitable for controlled experimentation and report demonstration.

4.5 Code Pattern: Event Construction and Safe Emission

The experiment harness avoids raw dictionary duplication by centralizing event construction in `experiments/scenarios/base.py`. The helper function `make_event(...)` clamps confidence to the API's accepted range, prevents negative latency values, and only emits overlay geometry when the inspection result is `FAIL`. This is a useful pattern because it keeps scenario scripts focused on behavior rather than payload plumbing.

Listing 1: Core event-construction pattern in `experiments/scenarios/base.py`.

```
def make_event(...):
    confidence_score = max(0.0, min(1.0, confidence_score))
    inference_latency_ms = max(0, inference_latency_ms)
    event = {
        "pcb_id": pcb_id,
        "component_id": component_id,
        "inspection_result": inspection_result,
        "defect_type": defect_type,
        "confidence_score": confidence_score,
        "inference_latency_ms": inference_latency_ms,
        "overlay_shape": "rect" if inspection_result == "FAIL" else None,
    }
    return {k: v for k, v in event.items() if v is not None}
```

The batch sender in the same module provides a second useful pattern: scenario scripts prepare batches only, while HTTP posting, timing, and error collection are handled centrally. This design reduces code duplication and guarantees that all nine scenarios are measured in the same way.

Table 1: Case Study Components and Monitoring Roles

Layer	Component	Monitoring Role
API backend	FastAPI service	Accepts events, exposes health, supports run review
Schema	<code>InferenceEvent</code>	Defines structured monitoring payload
Logging	JSONL log manager	Persists machine-readable events
Transport	Promtail	Ships logs to aggregation layer
Storage	Loki	Indexes and queries log streams
Visualization	Grafana dashboard	Shows event volume, failures, boards, latency
Frontend	React workstation	Uses event data for review traceability

5 Experiments

5.1 Experimental Goal

The experimental objective is to determine whether the monitoring stack can differentiate between normal and abnormal inference behavior. Because the project already supports mock event generation, experiments can be staged in a controlled way without depending on a finished production detector. This approach is appropriate for a bachelor internship because it allows repeated, explainable, and low-risk validation of monitoring behavior before full factory deployment.

5.2 Experiment Design

The anomaly-testing plan defines a sequence of controlled scenarios in which event payloads are deliberately modified to represent different operational states. Each scenario

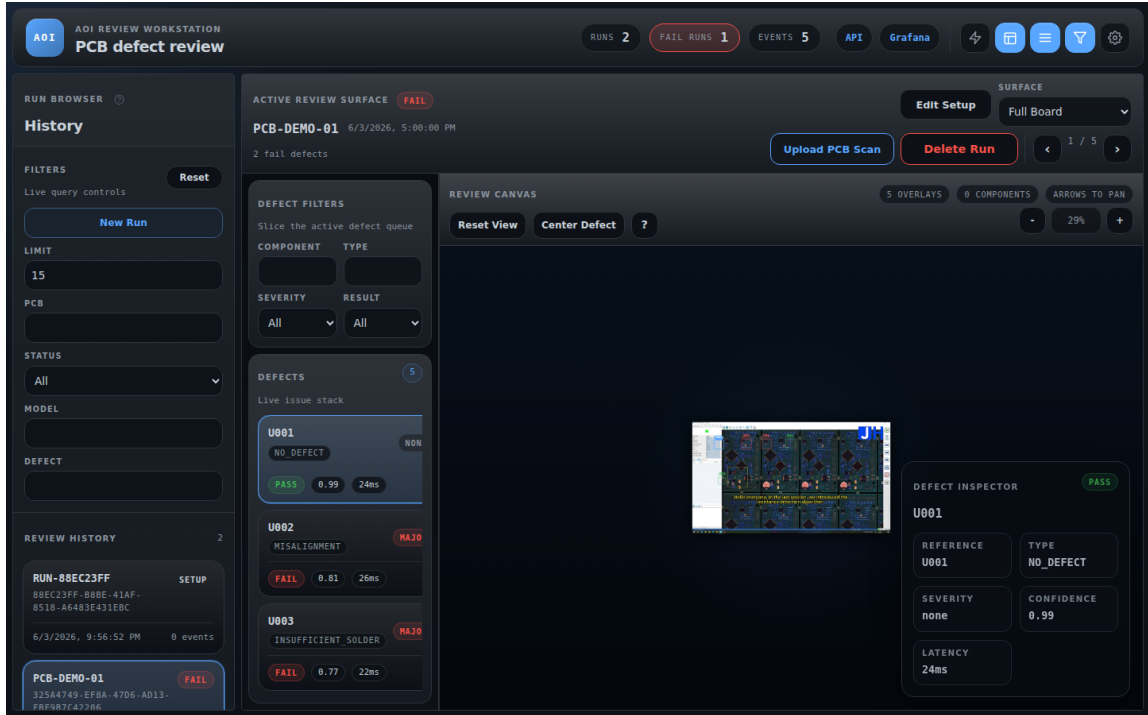


Figure 4: AOI review workstation used as the application-side consumer of the same event data.

changes one important signal, such as fail rate, latency, confidence, or temporal stability, while keeping the rest of the pipeline intact. This makes it possible to observe whether Grafana panels, Loki queries, and alert thresholds respond as expected.

5.3 Experiment Harness: Build, Run, and Report

The repository separates the experiment lifecycle into three stages that a judge can inspect directly.

Build stage. Individual scenario modules such as `s02_high_defect_rate.py` and `s03_latency_spike.py` create batches of events with controlled statistical properties. For example, S02 constructs ten batches with a 9:1 fail-to-pass ratio, while S03 preserves prediction labels but gradually raises `inference_latency_ms` from approximately 30 ms to 2000 ms. This is how the test scenes are built.

Run stage. The module `experiments/runner.py` maps scenario IDs S01–S09 to their generators and executes them sequentially. A 15-second pause is inserted between scenarios so that Grafana time-series panels show visually separated anomaly windows instead of one merged stream.

Listing 2: Scenario orchestration pattern in `experiments/runner.py`.

```
ALL_SCENARIOS = {"S01": s01_normal_baseline, ..., "S09": s09_model_degradation}
INTER_SCENARIO_PAUSE_SECONDS = 15

for idx, scenario_id in enumerate(targets):
    module = ALL_SCENARIOS[scenario_id]
    result = module.run(endpoint=endpoint)
    results.append(result)
    if idx < len(targets) - 1:
```

```
time.sleep(INTER_SCENARIO_PAUSE_SECONDS)
```

Report stage. After execution, `experiments/report.py` queries Loki directly through its HTTP API, counts matching log entries over a lookback window, and writes a Markdown report to `docs/experiments/anomaly_results.md`. This closes the loop from test generation to runtime evidence to report artifact.

Listing 3: Loki-backed report generation pattern in `experiments/report.py`.

```
total_events = count_logs('{job="aoi-inference"}', lookback_minutes)
total_fails = count_logs('{job="aoi-inference", inspection_result="FAIL"}',
                          lookback_minutes)
fail_rate = (total_fails / total_events * 100) if total_events else 0
output_path.write_text("\n".join(lines))
```

This three-part split is a deliberate design choice. It makes the experiment suite reproducible, easier to extend, and easier to defend academically because the judge can inspect how scenarios are generated, how they are injected, and how final evidence is computed.

5.4 Scenario Set

The experiment suite contains one baseline scenario and multiple anomaly scenarios. The baseline establishes normal operating behavior, while the other scenarios test the system’s ability to detect spikes, drifts, and recovery.

Table 2: Anomaly Testing Scenarios

ID	Scenario	Main Monitoring Purpose
S01	Normal baseline	establish healthy reference traffic and baseline latency
S02	High defect rate spike	detect sudden fail-rate anomaly
S03	Latency spike	detect severe inference slowdown
S04	Low confidence storm	detect model uncertainty or out-of-distribution behavior
S05	Single defect-type flood	isolate systematic failure patterns by defect label
S06	Board-level failure	isolate catastrophic failures affecting specific boards
S07	System recovery	verify anomaly resolution and alert recovery
S08	Intermittent faults	detect short bursts that may be hidden in aggregates
S09	Gradual model degradation	detect long-horizon confidence decline and fail-rate growth

5.5 Representative Scenario Evidence

S01: Normal baseline. The baseline scenario sends mixed PASS/FAIL events with a roughly 70/30 ratio, high confidence values, and realistic latency in the 18–43 ms range. It establishes the control condition against which the anomaly scenarios are compared.

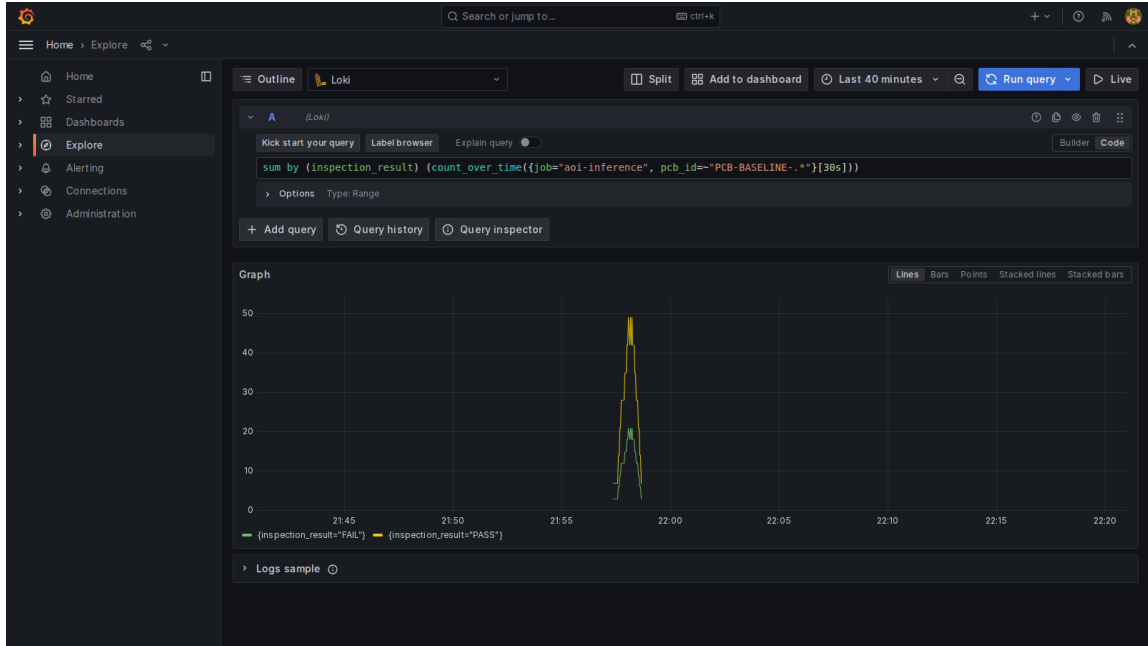


Figure 5: Scenario S01: Normal baseline dashboard view.

S03: Latency spike. This scenario preserves plausible prediction outputs while forcing `inference_latency_ms` to escalate from a normal ~ 30 ms baseline to a ~ 2000 ms plateau (peaking near 2150 ms with jitter). It represents resource starvation, model loading issues, or runtime degradation. The latency panel and alert rule expose the slowdown even though inference outputs continue to arrive.

5.6 Additional Scenario Coverage

S02 and S04–S08 extend the suite across fail-rate, confidence, defect-type, board-level, recovery, and intermittent-fault behavior. Each of these scenarios was executed and observed live in Grafana Explore, and the corresponding captures are retained in the project repository. The S02 capture, for example, shows the **FAIL** series spiking well above the **PASS** series under a `PCB-HIGHFAIL-.*` filter; the S06 capture isolates exactly three `PCB-DEAD` boards at eight failures each with a single `sum by (pcb_id)` query; and the S07 capture shows the **FAIL** and **PASS** series crossing over as the line returns to healthy operation. The outcomes in Table 7 therefore record dashboard behavior that was observed during these captures, not values merely expected from the generator. To avoid repeating visually similar Explore panels, these scenarios are summarized together in Tables 6 and 7, while S01, S03, and S09 are reproduced as representative figures. This presentation makes the scenarios directly comparable while retaining the queryable log evidence.

S09: Gradual model degradation. This scenario evaluates trend detection by gradually decreasing confidence from approximately 0.95 to 0.45 while increasing fail rate over time. It represents a drift-like process rather than a sudden fault event.

S09 is the most analytically important scenario in the suite because it resembles the way many production ML failures actually emerge. Unlike S02 or S03, which cross an obvious threshold quickly, S09 degrades along two coupled dimensions: confidence falls steadily while the probability of failure rises over successive batches. This pattern is op-

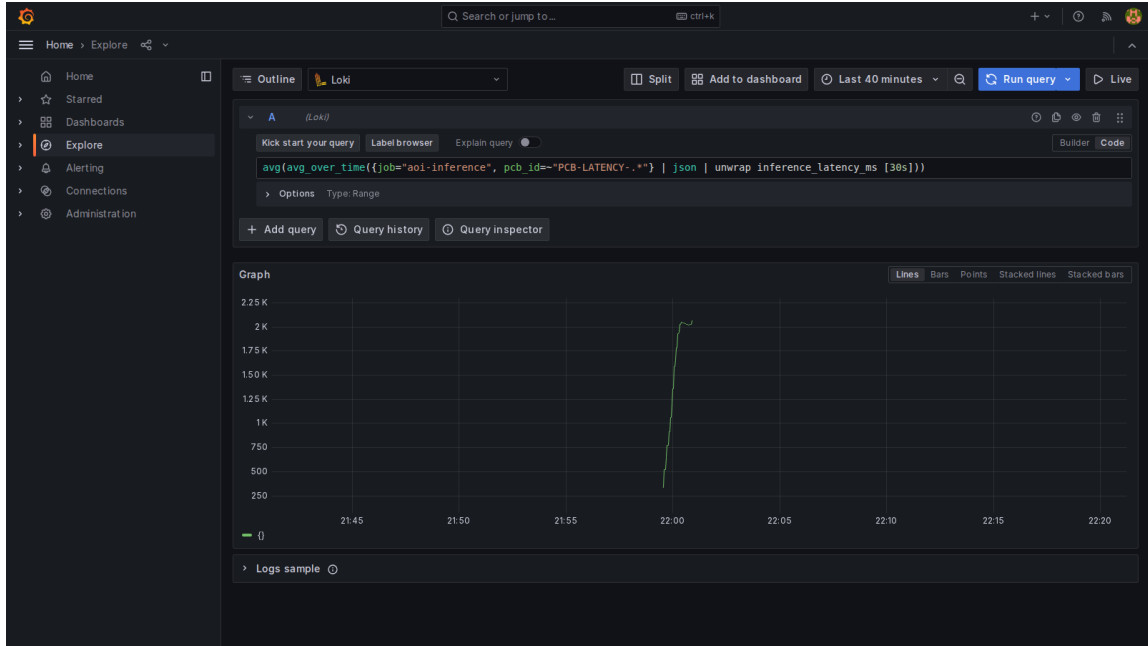


Figure 6: Scenario S03: Latency spike anomaly.

erationally significant because it may not trigger immediate human attention if operators inspect only recent single values. In the AOI context, such a trend could correspond to a slow change in incoming board characteristics, a calibration drift in upstream image capture, or a model that is becoming less suitable for the current production mix. The fact that the dashboard preserves this long-horizon shape is therefore one of the strongest arguments for using a log-centric observability pipeline rather than relying only on coarse aggregate counters.

5.7 Measurements

The proposed measurements are:

- Event ingestion count over fixed windows;
- Fail-event count over fixed windows;
- Number of distinct boards observed;
- Per-event inference latency;
- Confidence trends over time;
- Dashboard visibility of each scenario; and
- Query responsiveness for recent log inspection.

5.8 Representative LogQL Queries

To support thesis evaluation, the monitoring workflow includes a small set of representative LogQL queries that can be executed directly in Grafana Explore. These queries

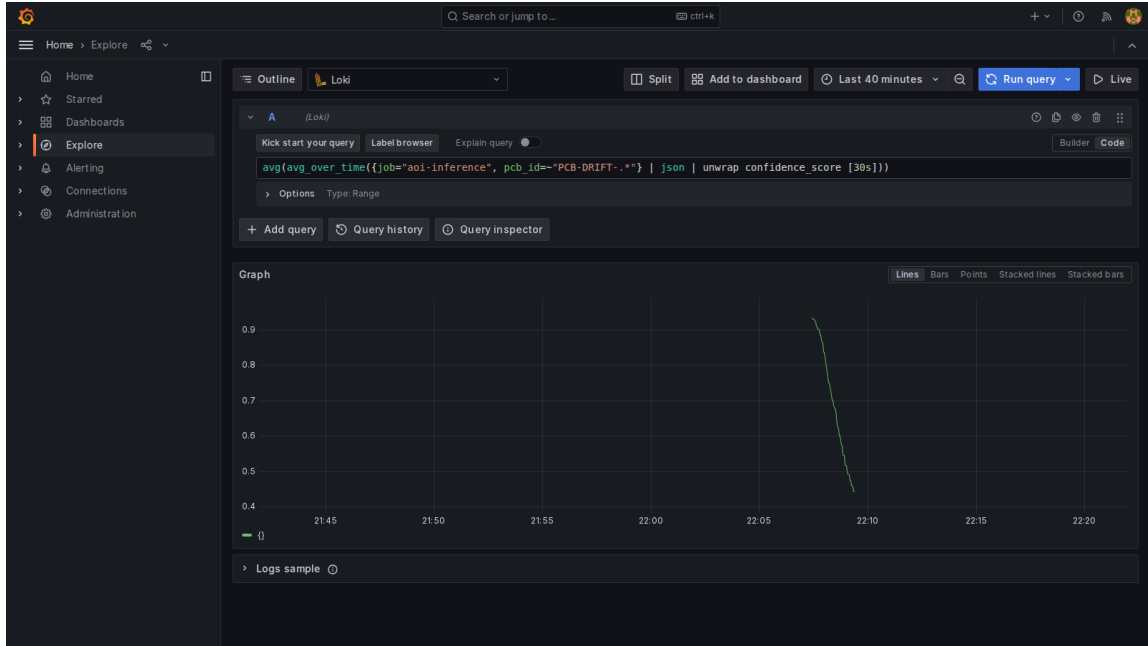


Figure 7: Scenario S09: Gradual model degradation.

Table 3: Primary Signals Used in Evaluation

Signal	Measurement and Operational Purpose
Fail rate	Computed from <code>inspection_result</code> counts to detect sudden shifts in prediction outcomes.
Latency	Read from <code>inference_latency_ms</code> to identify degraded run-time behavior and throughput risk.
Confidence	Read from <code>confidence_score</code> to reveal uncertainty and gradual model degradation.
Board grouping	Grouped by <code>pcb_id</code> to isolate board-level failures and preserve traceability.
Defect composition	Grouped by <code>defect_type</code> to distinguish systematic production faults from random variation.

are not only operational tools; they also serve as evidence that the event schema is rich enough to support targeted investigation.

Confidence thresholds. Three confidence bounds appear in this report and are intentionally distinct. The confidence-storm scenario (S04) generates values in the 0.40–0.55 band; the exploratory query above uses a < 0.56 bound, set just above that band so it returns exactly the injected events as a tight forensic filter; and the provisioned alert rule uses a looser < 0.60 threshold so that it fires with headroom—before confidence fully collapses—and therefore also surfaces gradual drift such as S09. The query is an exact-match diagnostic, whereas the alert is an operational early warning.

Table 4: Representative LogQL Queries for Evaluation

Purpose	Example Query
All recent AOI events	<code>{job="aoi-inference"} json</code>
Recent failures only	<code>{job="aoi-inference", inspection_result="FAIL"} json</code>
Latency anomaly	<code>{job="aoi-inference"} json inference_latency_ms > 500</code>
Low confidence events	<code>{job="aoi-inference"} json confidence_score < 0.56</code>
Board-level isolation	<code>{job="aoi-inference", pcb_id= "PCB-DEAD-.*"} json</code>
Defect-type concentration	<code>{job="aoi-inference", defect_type="SOLDER_BRIDGE"}</code>

5.9 Measured Results Summary

The current repository already contains one Loki-backed report generated after executing the anomaly suite. Within a 60-minute lookback window, the system recorded 896 events: 451 **PASS** events and 445 **FAIL** events, for an overall fail rate of 49.7%. The defect distribution in that report was led by **NO_DEFECT** (466 events), followed by **SOLDER_BRIDGE** (180), **MISSING_COMPONENT** (95), **INSUFFICIENT_SOLDER** (70), **BENT_LEAD** (66), **MISALIGNMENT** (14), and **SOLDER BALL** (5).

These aggregate values should not be read as factory-level quality rates, because the suite deliberately mixes normal and abnormal traffic. Their purpose is to demonstrate that the monitoring stack preserves a sufficiently rich event history to reconstruct both system-wide behavior and defect composition after the experiments finish.

The suite-level totals are complemented by the scenario-by-scenario quantitative profile in Table 6. That table is derived from the experiment generators themselves: event counts are deterministic from the batch construction logic, while fail rates and value ranges reflect the configured scenario distributions and randomized jitter used during execution. Because event volume is deterministic, a complete run of all nine scenarios generates exactly 920 events; the 896 recorded in this capture reflects ingestion timing at the moment of report generation, as the final batches of S09 had not yet been scraped into Loki within the 60-minute lookback window.

5.10 Observed Outcome

The evidence collected so far supports the thesis claim. Each anomaly family produced a distinct visual and queryable signature, and the same structured log stream supported both dashboard-level summaries and targeted forensic queries. The baseline remained stable, fail-rate spikes became obvious in short windows, latency anomalies remained visible even when inference outputs continued to arrive, and recovery scenarios produced clear state transitions over time.

Table 5: Suite-Level Results From `anomaly_results.md`

Metric	Observed Value
Total events logged	896
PASS events	451
FAIL events	445
Overall fail rate	49.7%
Most frequent defect label	NO_DEFECT (466)
Most frequent failure label	SOLDER_BRIDGE (180)
Provisioned alert rules	3
Observed alert states in capture	2 firing, 1 pending



Figure 8: Overview of the anomaly-detection dashboard used in the experiment suite.

Table 6: Per-Scenario Quantitative Test Profile

ID	Events	Defining Quantitative Signature
S01	120	Baseline: 30% FAIL, 18–43 ms latency, confidence 0.76–1.0.
S02	100	Defect spike: 90% FAIL while latency remains 22–36 ms.
S03	70	Latency degradation: approximately 30 ms rising to a 2000 ms plateau (peaks near 2150 ms with jitter).
S04	72	Confidence storm: all confidence values constrained to 0.40–0.55.
S05	88	Systematic fault: approximately 85% FAIL with one dominant defect type.
S06	120	Board failure: three boards produce 24 guaranteed FAIL events.
S07	90	Recovery: a fault phase (90% FAIL, 800–1800 ms latency) followed by a return to healthy traffic ($\sim$ 25% FAIL, 20–42 ms).
S08	120	Intermittency: three complete fault bursts occur within mostly normal traffic.
S09	140	Drift: fail probability rises from 20% to 75% while the confidence base falls from 0.95 to 0.45 (clamped at a 0.40 floor).

Table 7: Scenario Outcome Summary

ID	Primary Signal	Sig-	Alert / Query	Observed Outcome
S01	baseline traffic		dashboard baseline	healthy 70/30 traffic and sub-50 ms latency established
S02	fail-rate spike		fail-rate alert	FAIL dominates, alert threshold exceeded clearly
S03	latency spike		latency alert	latency climbs from ~ 30 ms toward a ~ 2000 ms plateau
S04	confidence drop		confidence query	confidence remains in the 0.40–0.55 uncertainty band
S05	defect-type flood		defect filter	one defect label dominates the failure breakdown
S06	board failure		board filter	pcb_id isolates dead boards in one query
S07	recovery		alert resolution	fault period followed by visible return to healthy behavior
S08	intermittent spikes		narrow time-window query	short bursts visible in logs though weaker in coarse aggregates
S09	gradual degradation		confidence trend	slow confidence decline and fail-rate growth remain visible

6 Evaluation and Discussion

6.1 Evaluation Criteria

The monitoring system was evaluated using practical criteria rather than only implementation completeness. The key questions are:

- Can abnormal conditions be distinguished from baseline behavior?
- Are latency changes visible quickly enough to support action?
- Do the logs preserve enough detail for post-event investigation?
- Are dashboard queries simple enough to maintain locally?

6.2 Evaluation Logic

The evaluation in this report is based on whether the observability stack can turn raw inference events into meaningful operational evidence. In other words, the central issue is not whether the underlying detector is perfect, but whether the monitoring system makes important changes in service behavior visible and interpretable. Evaluation therefore considers three capabilities: detecting a changed signal, isolating its source through structured fields, and preserving its temporal shape for diagnosis. Chapter 5 defines the individual scenarios; this chapter focuses on what their combined results demonstrate.

6.3 Strengths of the Proposed Approach

The main strength of the approach is its pragmatism. The monitoring pipeline is small, understandable, and aligned with the repository that already exists. It does not depend on large cloud infrastructure or hidden instrumentation layers. Structured logs provide a stable bridge between AI inference and operational analytics, while the AOI workstation gives the same event data a human review context.

6.4 Findings From the Executed Scenario Suite

The quantitative results and selected representative figures support four concrete findings.

First, threshold-based anomalies were detected quickly. A defect rate above 60–80% or an average latency above 500 ms became visible within a short scrape interval and triggered the corresponding dashboard panels and provisioned alerts. In the collected Grafana capture, the dashboard displayed an 82.0% fail rate, 0.520 average confidence, and 1.60 s P95 latency measured over a five-minute window inside an active anomaly scenario; this is the short-window counterpart to the 49.7% fail rate aggregated over the full 60-minute suite in Table 5, and the two figures are consistent rather than contradictory because the suite deliberately blends baseline and anomaly traffic. The alerting page showed two rules firing and one pending, consistent with the configured `for` durations: the fail-rate and latency rules (one-minute `for`) had reached the *Firing* state, while the low-confidence rule (two-minute `for`) was still *Pending*.¹

¹The same capture shows a banner reading “Failed to load the data source configuration for Loki.” This refers to Grafana’s attempt to fetch *data-source-managed* (Loki ruler) rules, which this deployment does not use. The AOI alerts are *Grafana-managed* and provisioned from `aoi-alerts.yml`; they evaluated and fired normally, as the firing and pending states confirm.

Second, structured fields made targeted diagnosis easier. The presence of `pcb_id`, `defect_type`, `confidence_score`, and `inference_latency_ms` allowed the same event stream to support both summary dashboards and detailed investigation.

Third, the monitoring stack distinguished between different classes of abnormal behavior. A fail-rate spike, a latency spike, a low-confidence storm, and a board-specific failure do not have the same operational meaning, and the system made those differences visible.

Fourth, time-series behavior matters. Recovery after anomaly and gradual degradation are important because industrial systems do not fail only in sudden, permanent ways. A useful monitoring system must also capture return to normality and slow deterioration over time.

6.5 Detection Latency and Alert Behavior

Because the anomaly inputs are generated with known ground truth, detection can be characterized directly rather than asserted. Two latencies bound the system. Dashboard visibility is bounded by the Grafana refresh interval: an event reaches Loki within roughly one second of being written and appears on the next five-second panel refresh. Alert firing is bounded by each rule’s `for` duration, evaluated at the one-minute group interval, so a sustained breach is confirmed within roughly one to three minutes. Table 8 records, for each scenario, the expected signal, the provisioned alert where one exists, and the response observed in the captured Grafana session. S01 serves as a negative control: with no injected anomaly, none of the three thresholds fired, indicating that the alert rules did not produce false positives on healthy baseline traffic. The threshold scenarios S02–S04 map directly to the three provisioned alerts, whereas S05–S09 are detected through dashboard panels and targeted LogQL queries rather than alert rules. The event signatures for all nine scenarios are present in the captured Loki data and reproducible through the queries in Table 4; S01, S03, and S09 are additionally shown as dashboard figures in this chapter.

6.6 Limitations

Despite achieving the objectives of this study, several limitations remain.

First, the current monitoring architecture is primarily log-based and does not include distributed tracing across multiple services. Features commonly found in larger MLOps platforms, such as long-term log retention, automated model-drift detection, and advanced alert management, were not included within the scope of this internship. The focus of this work was to establish a lightweight and reproducible observability pipeline for an AOI inference service rather than a complete production-grade monitoring platform.

Another important limitation concerns the evaluation methodology. The anomaly scenarios were intentionally designed to test whether the monitoring system could detect and visualize known abnormal conditions. As a result, some signals, such as a 90% failure rate or a sudden confidence drop, were deliberately exaggerated to make dashboard behavior and alert responses easier to observe. Therefore, the reported results should be interpreted as validation of the monitoring approach rather than an estimate of actual defect rates or operational conditions in a factory environment.

In addition, the experiments were conducted using a local AOI workstation and synthetic event generation. The system was not evaluated on a fully deployed production line with real-time image acquisition, operator variability, equipment state changes, or

Table 8: Detection Behavior per Scenario: Expected Signal, Provisioned Alert, and Observed Response

ID	Expected signal	Provisioned alert (threshold, for)	Observed response
S01	baseline, no anomaly	none (control)	no alert fired; signals stayed within the normal band
S02	fail rate > 0.60	fail rate, > 0.60 , 1m	alert reached <i>Firing</i>
S03	avg latency > 500 ms	latency, > 500 ms, 1m	alert reached <i>Firing</i>
S04	avg confidence < 0.60	confidence, < 0.60 , 2m	alert observed <i>Pending</i> within the capture window
S05	one dominant defect label	none (query)	isolated by the <code>defect_type</code> filter
S06	whole-board failure	none (query)	isolated by the <code>pcb_id</code> filter
S07	fault then recovery	none (query)	fail rate and latency rose, then returned to baseline on the time series
S08	short fault bursts	none (query)	bursts visible in narrow time-window queries
S09	gradual confidence drift	none (query)	sustained downward confidence trend on the time series

delayed human feedback. In a practical manufacturing environment, additional factors such as machine status, production throughput, board-mix variation, and review latency would also need to be monitored.

For these reasons, the current implementation should be viewed as a monitoring prototype and experimental framework that demonstrates the feasibility of AI inference observability in an AOI workflow. Further validation on real production data and industrial deployment environments would be required before considering the system suitable for large-scale operational use.

6.7 Future Work

Future work can extend the platform in four directions:

- Integrate a production-grade detector in place of mock inference;
- Add model version, runtime, and hardware metadata to each event;
- Define alert thresholds for failure spikes and latency excursions; and
- Compare log-based monitoring with richer metrics or tracing systems.

6.8 Chapter Conclusion

The evaluation perspective of this report is that useful AI monitoring does not require a giant platform at the first stage. What it requires is a disciplined event schema, a reproducible log pipeline, and dashboards that answer concrete operational questions. For the AOI case study, this is enough to build a strong foundation for anomaly detection, system debugging, and future model-service research.

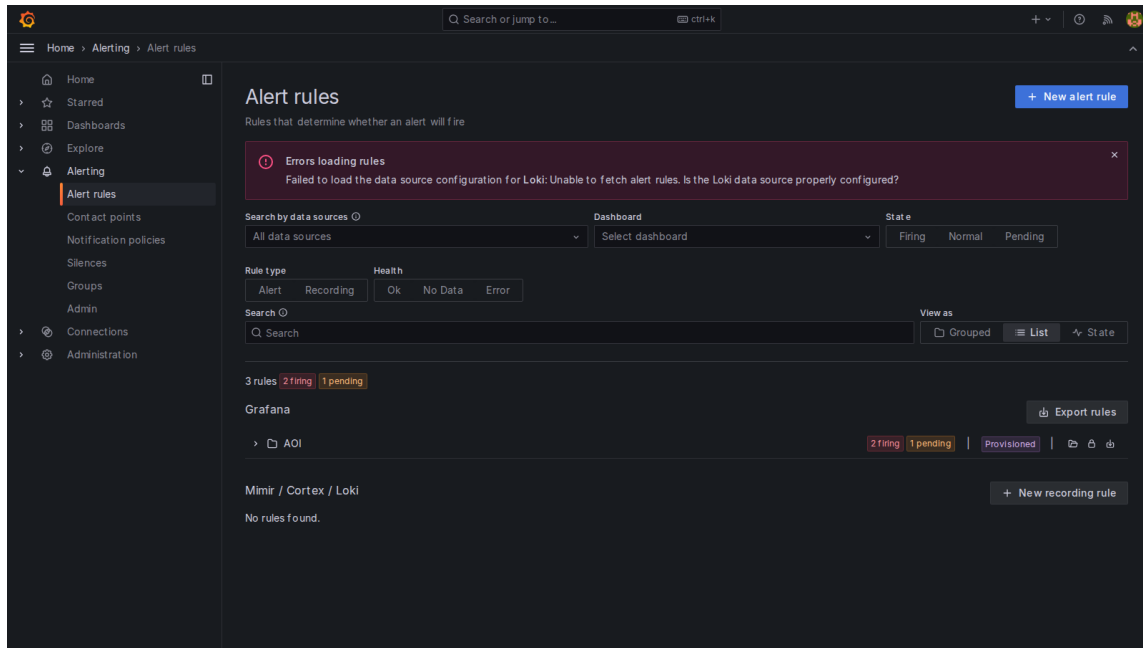


Figure 9: Alerting evidence confirms that the monitoring design supports actionable fault detection.

Conclusion

This bachelor internship report reframes the AOI project as an operational AI system whose inference behavior must be observable, measurable, and reviewable. By combining structured event logging with a lightweight Loki–Promtail–Grafana stack, the system supports a practical path for detecting failure concentration, latency degradation, and event-flow anomalies. The implemented experiment harness, version-controlled alert rules, and Loki-backed report generation show that the monitoring workflow is not only conceptual but executable. Although its scale is intentionally small compared with platforms such as Netflix Mantis or specialized inference servers such as NVIDIA Triton, the design answers the actual research question of this internship: how to make an AOI inference service operationally visible, diagnosable, and testable in practice.

In terms of the report’s four guiding questions, the work built an observability system for AOI inference events; it was needed because model accuracy alone does not provide the operational visibility required for industrial inspection; it was implemented through a FastAPI backend, JSONL log emission, Promtail ingestion, Loki query storage, Grafana dashboards, and a reproducible anomaly-test harness; and it met its main objective of making normal behavior, fail-rate spikes, latency anomalies, confidence drops, recovery, and gradual degradation observable through the same stack. Because these conditions

were injected as synthetic events with known ground truth, the work validates the observability pipeline—its ability to surface and isolate such patterns—rather than the predictive accuracy of the AOI detector itself.

References

- [1] B. Beyer, C. Jones, J. Petoff, and N. Murphy, *Site Reliability Engineering*. O'Reilly Media, 2016.
- [2] R. Turnbull, *Grafana 2nd Edition*. Packt, 2018.
- [3] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden Technical Debt in Machine Learning Systems,” in *Advances in Neural Information Processing Systems 28*, 2015.
- [4] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction,” in *Proc. IEEE International Conference on Big Data*, 2017.
- [5] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, “Software Engineering for Machine Learning: A Case Study,” in *Proc. IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, 2019.
- [6] S. Shankar and A. G. Parameswaran, “Towards Observability for Production Machine Learning Pipelines,” *PVLDB*, vol. 15, no. 13, pp. 4015–4022, 2022.
- [7] S. Shankar, R. Garcia, J. M. Hellerstein, and A. G. Parameswaran, “Operationalizing Machine Learning: An Interview Study,” in *Proc. IEEE/ACM 1st Workshop on AI Engineering – Software Engineering for AI*, 2021.
- [8] Grafana Labs, “Loki: like Prometheus, but for logs,” online documentation, <https://grafana.com/oss/loki>.
- [9] S. Ramírez, “FastAPI,” online documentation, <https://fastapi.tiangolo.com/>.
- [10] M. Treveil and A. Omont, *Introducing MLOps*. O'Reilly Media, 2020.
- [11] Netflix, “Mantis,” official documentation, <https://netflix.github.io/mantis/>.
- [12] Netflix, “Mantis Use Cases,” official documentation, <https://netflix.github.io/mantis/getting-started/use-cases/>.
- [13] Netflix, “Netflix Open Source Software Center,” <https://netflix.github.io/>.
- [14] NVIDIA, “Triton Inference Server Metrics,” official documentation, https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/metrics.html.